

LABSHEET-4**NAME- Sunny****CU240251760****SEC-B****ROLL NO- 58****2. Import the required libraries (Pandas, Matplotlib, and Seaborn) and load the cleaned dataset.**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.set_style("whitegrid")
```

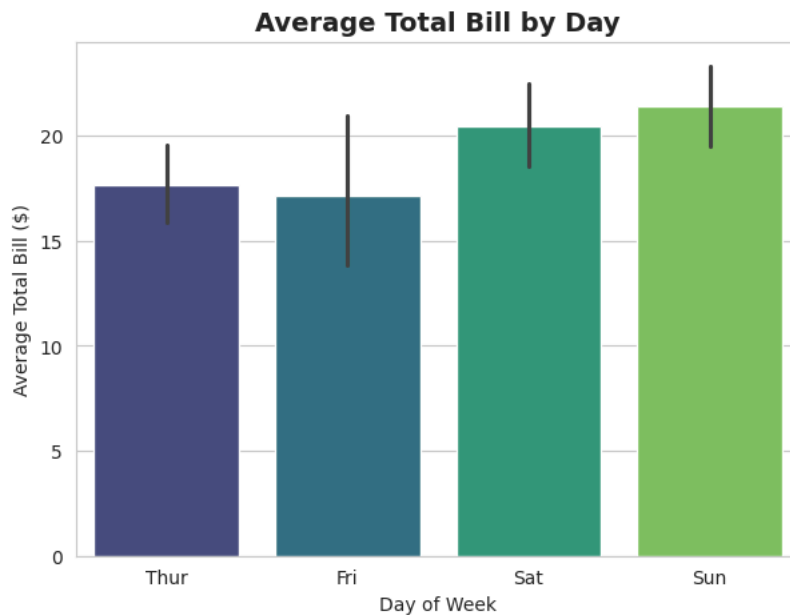
Load dataset

```
tips = sns.load_dataset("tips")
print(tips.head())
print(tips.info())
```

```
   total_bill  tip  sex smoker  day  time  size
0    16.99   1.01 Female    No  Sun  Dinner     2
1    10.34   1.66  Male    No  Sun  Dinner     3
2    21.01   3.50  Male    No  Sun  Dinner     3
3    23.68   3.31  Male    No  Sun  Dinner     2
4    24.59   3.61 Female    No  Sun  Dinner     4
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0  total_bill  244 non-null      float64
1  tip         244 non-null      float64
2  sex         244 non-null      category
3  smoker      244 non-null      category
4  day         244 non-null      category
5  time        244 non-null      category
6  size        244 non-null      int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
None
```

3. Create bar charts to compare categorical variables such as sales by region, genre distribution, or product categories.

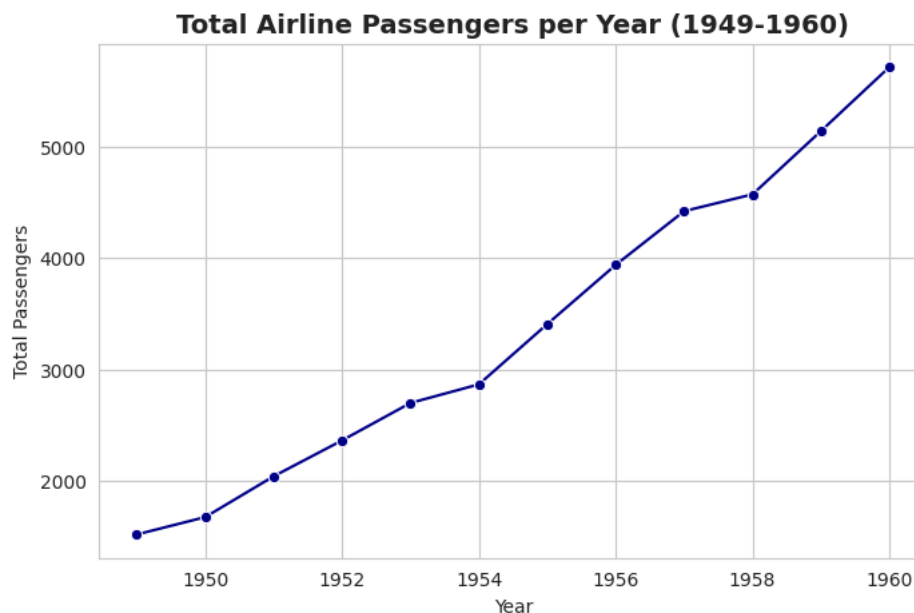
```
plt.figure(figsize=(7, 5))
sns.barplot(data=tips, x="day", y="total_bill", hue="day", palette="viridis", legend=False)
plt.title("Average Total Bill by Day", fontsize=14, fontweight="bold")
plt.xlabel("Day of Week")
plt.ylabel("Average Total Bill ($)")
plt.show()
```



4. Generate line charts to analyze trends over time, such as monthly sales, yearly movie releases, or daily COVID-19 cases

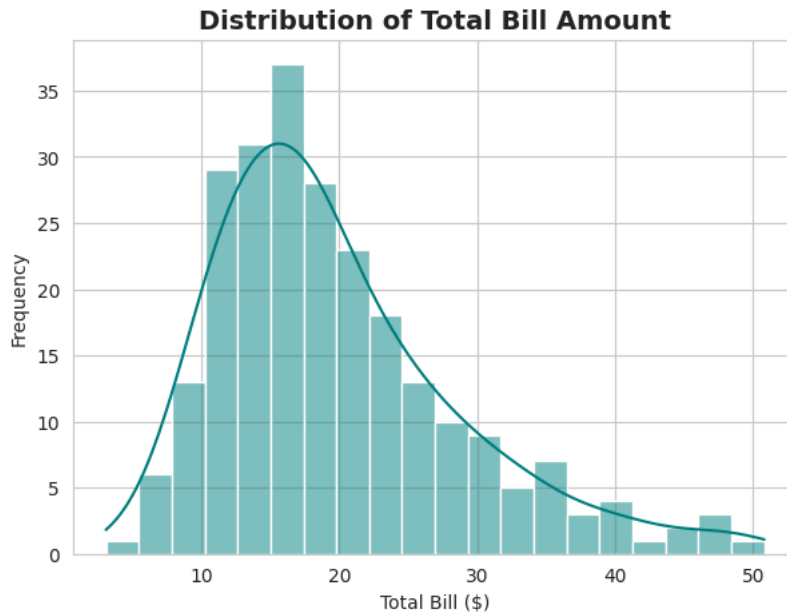
```
flights = sns.load_dataset("flights")
yearly = flights.groupby("year")["passengers"].sum().reset_index()

plt.figure(figsize=(8, 5))
sns.lineplot(data=yearly, x="year", y="passengers", marker="o", color="darkblue")
plt.title("Total Airline Passengers per Year (1949-1960)", fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Total Passengers")
plt.show()
```



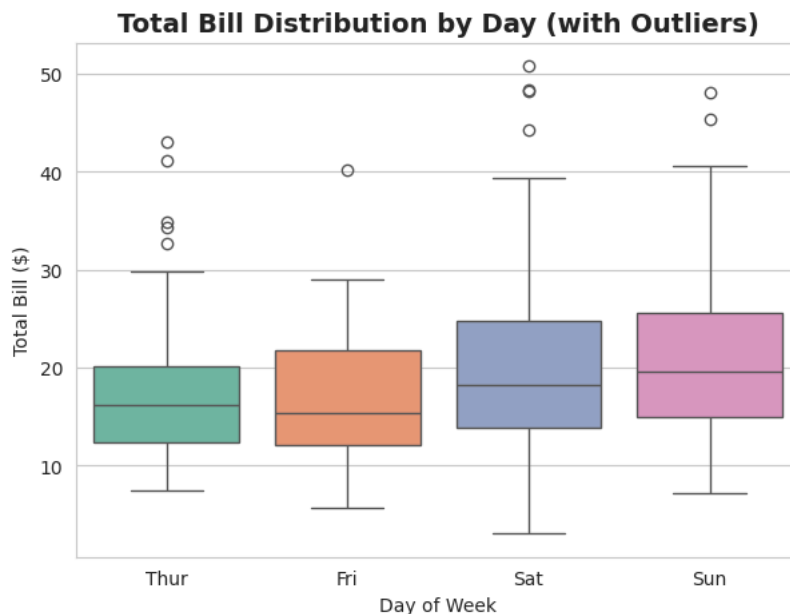
**5. Plot histograms to study the distribution of numerical variables such as sales, ratings, or customer age. **

```
plt.figure(figsize=(7, 5))
sns.histplot(tips["total_bill"], bins=20, kde=True, color="teal")
plt.title("Distribution of Total Bill Amount", fontsize=14, fontweight="bold")
plt.xlabel("Total Bill ($)")
plt.ylabel("Frequency")
plt.show()
```



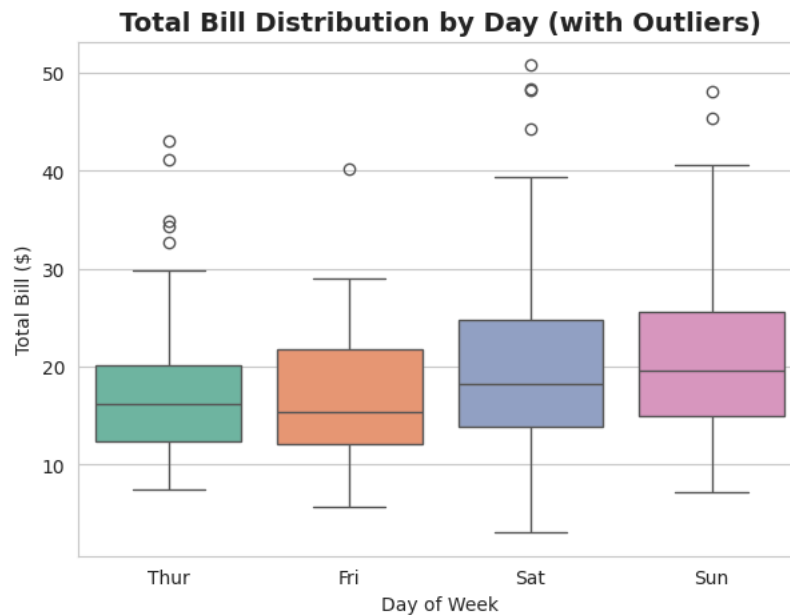
****6. Create box plots to identify outliers and compare distributions across different categories. ****

```
plt.figure(figsize=(7, 5))
sns.boxplot(data=tips, x="day", y="total_bill", hue="day", palette="Set2", legend=False)
plt.title("Total Bill Distribution by Day (with Outliers)", fontsize=14, fontweight="bold")
plt.xlabel("Day of Week")
plt.ylabel("Total Bill ($)")
plt.show()
```



7. Generate scatter plots to analyze relationships between two numerical variables, such as advertising expenditure vs. sales or budget vs. revenue.

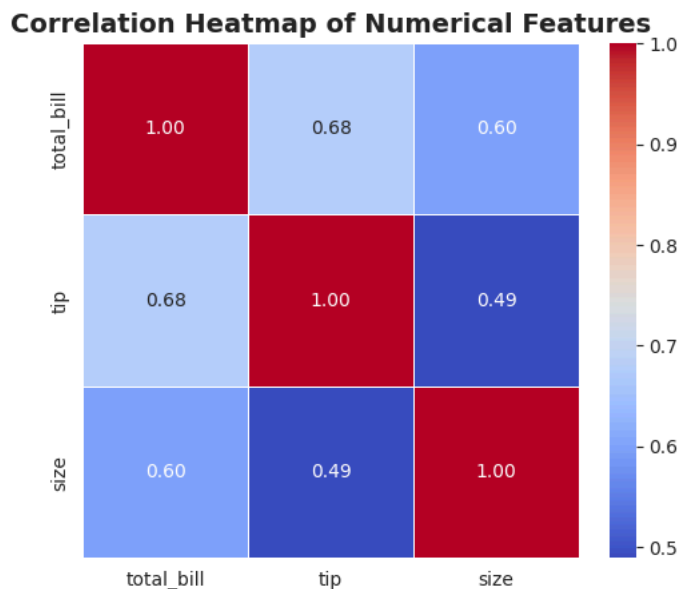
```
plt.figure(figsize=(7, 5))
sns.boxplot(data=tips, x="day", y="total_bill", hue="day", palette="Set2", legend=False)
plt.title("Total Bill Distribution by Day (with Outliers)", fontsize=14, fontweight="bold")
plt.xlabel("Day of Week")
plt.ylabel("Total Bill ($)")
plt.show()
```



8. Create a correlation heatmap using Seaborn to visualize relationships among numerical features.

```
numeric_cols = tips.select_dtypes(include="number")
corr = numeric_cols.corr()

plt.figure(figsize=(6, 5))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Features", fontsize=14, fontweight="bold")
plt.show()
```



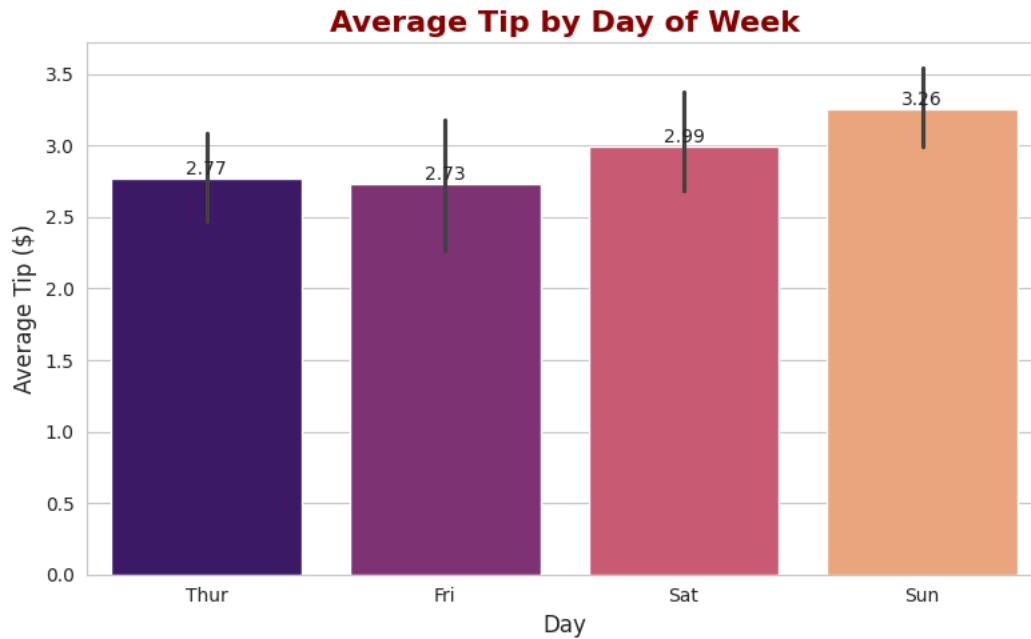
9. Customize the visualizations by adding titles, axis labels, legends, annotations, and appropriate color palettes to improve readability.

```
plt.figure(figsize=(8, 5))
ax = sns.barplot(data=tips, x="day", y="tip", hue="day", palette="magma", legend=False)
plt.title("Average Tip by Day of Week", fontsize=16, fontweight="bold", color="darkred")
plt.xlabel("Day", fontsize=12)
plt.ylabel("Average Tip ($)", fontsize=12)

# Add value annotations on top of each bar
for bar in ax.patches:
    height = bar.get_height()
    ax.annotate(f"{height:.2f}",
```

```
(bar.get_x() + bar.get_width() / 2, height),
ha="center", va="bottom", fontsize=10)

plt.tight_layout()
plt.show()
```



10. Interpret each visualization and prepare a summary highlighting key business insights obtained from the graphical analysis.

```
print("""
KEY INSIGHTS:
1. Weekends (Sat/Sun) show higher average total bills than weekdays.
2. Airline passenger traffic shows a clear upward yearly trend with seasonal peaks.
3. Total bill amounts are right-skewed - most bills are low-to-moderate with a few high outliers.
4. Box plots reveal outliers in total bill on certain days (e.g., large group bills).
5. Total bill and tip amount are strongly positively correlated (as bill increases, tip increases).
6. The correlation heatmap confirms 'total_bill' and 'tip' are the most correlated numeric features.
""")
```

```
KEY INSIGHTS:
1. Weekends (Sat/Sun) show higher average total bills than weekdays.
2. Airline passenger traffic shows a clear upward yearly trend with seasonal peaks.
3. Total bill amounts are right-skewed - most bills are low-to-moderate with a few high outliers.
4. Box plots reveal outliers in total bill on certain days (e.g., large group bills).
5. Total bill and tip amount are strongly positively correlated (as bill increases, tip increases).
6. The correlation heatmap confirms 'total_bill' and 'tip' are the most correlated numeric features.
```